GuildQuest
Final Updated Design Document

Sonja Muretta, Ryan Beck, Thang Nguyen

Table of contents

1. System overview
2. Architecture and Final Design
3. UML Diagram
4. Design Patterns
5. Reused Subsystems (Valid Reuse Checklist)
6. Mini-Adventures
7. How to Add a New Mini-Adventure
8. Design Retrospective
9. Gen-AI Deliverables
10. https://github.com/smuretta16/GuildQuest

1. System overview

The GuildQuest Mini-Adventure Environment (GMAE) is a two-player local gaming framework built on top of the GuildQuest world originally designed by Thang Nguyen and Sonja Muretta in earlier individual assignments. The system combines a campaign-management backend with a playable mini-adventure layer, all presented through a dark-themed Tkinter GUI.

<u>Goals:</u>

- Provide a reusable framework that makes adding new mini-adventures as simple as subclassing
one abstract class.
- Reuse at least two prior GuildQuest subsystems (realm model, item/inventory, quest events,
WorldClock, campaign sharing).
- Support exactly two local players with persistent profiles.
- Ship two complete, playable mini-adventures.

<u>Technology Stack:</u>

- Language: Python 3.11+
- GUI: Tkinter (standard library) — dark sidebar layout
- Game engine for Escort Mission: Pygame (real-time platformer)
- Persistence: JSON flat file (profiles.json)

2. Architecture and Final Design

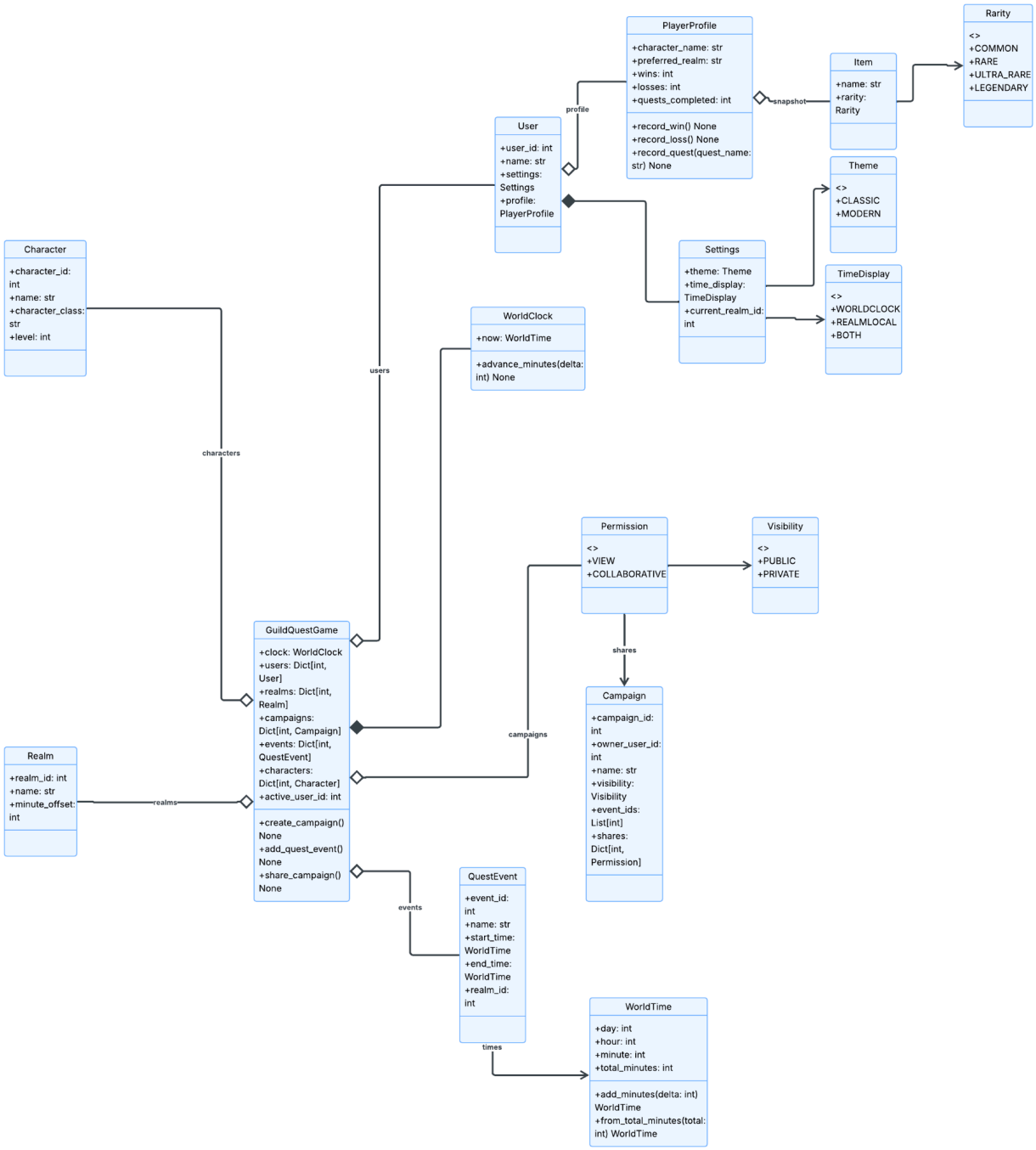
The system is split into three layers:

Layer	Files	Responsibility
GuildQuest Backend	guildquest.py	Domain models: User, Realm, Campaign, QuestEvent, Character, Item, WorldClock, PlayerProfile, Settings. Persistence
GMAE Core	mini_adventure.py game_context.py gmae.py	Abstract MiniAdventure interface, GameContext (shared state), GameController, MiniAdventureRegistry, ResultTracker.
Mini-Adventures	relic_hunt.py escort_mission.py	Concrete adventure implementations. Each subclasses MiniAdventure and is self-contained.
GUI	guildquest_gui.py	Tkinter sidebar app. Hosts all GuildQuest management pages, plus the Mini-Adventures launcher tab.

Key Architectural Decisions

• Single-file backend: guildquest.py is intentionally kept as one module to match the merged
individual-assignment approach. The GMAE layer is separate.
• Strategy pattern for mini-adventures: the GameController never imports RelicHunt or EscortMission directly — it only calls the MiniAdventure interface methods. New adventures require
zero changes to the controller.
• GameContext as a dependency-injection object: instead of passing the entire GuildQuestGame
into adventures, only the relevant subsystems (clock, realms, item pool) are exposed through
GameContext. This enforces the principle of least privilege.
• JSON persistence for profiles only: campaign/event data is seeded at startup and lives in memory. Only player profiles (wins, losses, quest history, inventory snapshots) are persisted
across sessions.

3. UML Diagram



This diagram is a class diagram of the Guildquest Backend (guildquest.py). It captures the core domain model with the following structure:

- **Enumerations**

    - Visibility, Permission, Theme, TimeDisplay, and Rarity are all shown as enum types at the top. These constrain the valid values for fields throughout the system (e.g., Campaign.visibility can only be public or private).
- **Time system**
    - WorldTime is a frozen value object holding day/hour/minute with arithmetic methods. WorldClock holds a single WorldTime instance and exposes advance_minutes(). The "times" label on the arrows shows WorldClock uses WorldTime. This is Sonja's time subsystem.
- **Realm**
    - A simple data object with a realm_id, name, and minute_offset. The "realms" label connects it to GuildQuestGame, showing the game holds a dictionary of realms. Settings also references a realm via current_realm_id
- **User/Settings/PlayerProfile**
    - The user owns a Settings object (theme, time display, current realm) and a PlayerProfile. PlayerProfile tracks wins, losses, quests completed, quest history, and inventory snapshot with methods for updating each. The "profile" and "snapshot" labels on arrows show these relationships
- **Campaign/QuestEvent**
    - Campaign holds a list of event_ids and a shares dictionary mapping user IDs to permission levels. QuestEvent holds start/end WorldTime values and a realm_id. The "campaigns" and "events" label show GuildQuestGame owns both dictionaries.
- **Character/Item**
    - Character holds character_id, name, class, and level. Item has name and rarity. The "characters" label connects Character to GuildQuestGame
- **GuildQuestGame**
    - The central coordinator class. It holds all the dictionaries (users, realms, campaigns, events, characters), the WorldClock, and the active_user_id. It exposes create_campaign(), add_quest_event(), and share_campaign() as the main operations.

4. Design Patterns

5.1 Strategy Pattern

- **Location**: mini_adventure.py + relic_hunt.py + escort_mission.py
- **Description**: MiniAdventure is the strategy interface. RelicHunt and EscortMission are concrete strategies. GameController is the context — it runs any adventure through the same 8-method interface without knowing which one it is. Adding a new adventure requires zero changes to the controller or GUI loop.
- **Justification**: Open/Closed Principle: the system is open for extension (new adventures) and closed for modification (controller unchanged). This directly satisfies the 'extensibility' requirement of the assignment.

5.2 Registry Pattern

- **Location**: gmae.py — MiniAdventureRegistry
- **Description**: All mini-adventures are registered in one place (GameController._register_adventures). The GUI reads the registry to build the adventure selection list. This decouples the menu from hard-coded adventure names.
- **Justification**: Makes the menu self-updating — teammates only add one line to register their adventure and it appears in the menu automatically.

5.3 Builder / Factory (light)

- **Location**: guildquest.py — GuildQuestGame.seed_data()
- **Description**: seed_data() constructs complex QuestEvent and Character objects with all required parameters in one place, inspired by Thang's Builder pattern from the individual assignment. The pattern is applied lightly (no formal Builder class) since Python dataclasses already provide clean construction.
- **Justification**: Keeps all seed/demo data in one method. Easy to swap out for a real database loader later without touching the rest of the code.

5.4 Input Validation (Security) [SECURITY]

- **Location**: relic_hunt.py — handle_input()
- **Description**: All player input is validated with a compiled regex before processing: re.compile(r'^[wasdWASD]$'). Player index is bounds-checked (must be 1 or 2). Turn order is enforced server-side — a player cannot submit input on the other player's turn.
- **Justification**: Prevents invalid moves from corrupting the game state. Also rejects injection-style inputs (e.g., 'DROP TABLE relics;') silently.

5.5 Path Traversal Prevention (Security) [SECURITY]

- **Location**: guildquest.py — save_profiles() / load_profiles()
- **Description**: Profile file operations are limited to a single known path (profiles.json). The load function silently ignores missing or malformed files rather than raising exceptions, preventing crashes from corrupt input. User IDs used as dict keys are always cast to int before use.

- **Justification**: Prevents a crafted profiles.json from causing path traversal or code injection through malformed keys or values

5. Reused Subsystems: Valid Reuse Checklist

The assignment requires the integration of at least two previously built GuildQuest subsystems. Our project integrates six subsystems drawn from Thang Nguyen's and Sonja Muretta's individual assignments.

Subsystem 1 — WorldTime / WorldClock (Time System)
Source: Sonja Muretta (individual assignments)

Where in repo: guildquest.py — WorldTime dataclass, WorldClock dataclass

What changed: Merged with Thang's version. Added a total_minutes computed property to make time arithmetic easier across the codebase. The core frozen dataclass design and add_minutes() method were kept exactly as originally designed. No structural changes to the time model itself.

Where it's used: Quest event scheduling (start_time and end_time on every QuestEvent are WorldTime objects), the advance world time feature in the Settings page, the sidebar clock display that updates live, time-range filtering for the day/week/month/year event views, and the EscortMission mini-adventure which advances the world clock by 10 minutes each time the carrier takes a hazard hit — directly tying real-time gameplay consequences back into the shared GuildQuest world clock.

Evidence: guildquest.py lines defining WorldTime and WorldClock; EscortMission.run_pygame_session() calls ctx.clock.advance_minutes(game.clock_penalty) on game end.

Subsystem 2 — Realm Model
Source: Thang Nguyen (individual assignments)

Where in repo: guildquest.py — Realm dataclass; GameContext.active_realm / get_realm_by_name()

What changed: The Realm class itself was kept unchanged (realm_id, name, description, minute_offset). The seed data was expanded from 2 realms (Avalon, Lunar Outpost) to 10 fully described realms: Avalon, Lunar Outpost, Shadowfen, Ironspire, Emberveil, Crystaldeep,

Thornwood, The Ashen Wastes, Skyreach, and Tidehollow — each with a unique description and time offset. GameContext was added as a new thin wrapper that exposes the realm dictionary to mini-adventures without exposing the entire GuildQuestGame.

Where it's used: Every QuestEvent is assigned to a realm via realm_id. The Realms page displays all realms with their offsets and descriptions. RelicHunt looks up the Avalon realm by name at init time and sets it as the active realm for the session. EscortMission looks up Shadowfen. The realm name is shown in the game window title and HUD so players know which GuildQuest location they are playing in.
Evidence: relic_hunt.py init() calls ctx.get_realm_by_name("Avalon"); escort_mission.py init() calls ctx.get_realm_by_name("Shadowfen").

Subsystem 3 — Campaign / QuestEvent System
Source: Both Thang Nguyen and Sonja Muretta

Where in repo: guildquest.py — Campaign dataclass, QuestEvent dataclass, GuildQuestGame campaign methods

What changed: Thang's version contributed event creation, realm assignment on events, and campaign deletion. Sonja's version contributed campaign visibility (PUBLIC/PRIVATE), ownership tracking, and the sharing/permissions model. These were merged into a single Campaign class. The event_ids list approach (Campaign stores IDs, events stored separately in a dict) was chosen as the canonical design. Four campaigns and ten quest events are now seeded at startup.

Where it's used: The Campaigns page, Quest Events page, and Dashboard upcoming events list all read from this subsystem. The time-range filtering (day/week/month/year views) uses QuestEvent.start_time compared against WorldClock.now. Campaign access control (owner / editor / viewer roles) is enforced via campaign_access() throughout the GUI.

Evidence: GuildQuestGame.list_visible_campaigns(), campaign_access(), print_events() all in guildquest.py; Campaigns and Events pages in guildquest_gui.py.

Subsystem 4 — Item / Inventory Model
Source: Thang Nguyen (individual assignments)

Where in repo: guildquest.py — Item dataclass, Rarity enum, Character.inventory list

What changed: The Item and Rarity classes were kept exactly as designed. Character.inventory remains a plain List[Item]. Pre-seeded inventory data was added for the four starter characters (Aldric, Seraphel, Vex, Brynn) to demonstrate the system at startup. The PlayerProfile.inventory_snapshot field was added as a new concept — a point-in-time copy of a player's collected items stored on their profile across sessions.

Where it's used: The Characters page displays each character's inventory with rarity labels. RelicHunt creates a new Item(name="Relic #N", rarity=RARE) each time a relic is collected and appends it to the player's inventory_snapshot via profile.update_snapshot(). EscortMission grants both players a Legendary Item("Escort Medal") on successful completion. This means every mini-adventure outcome leaves a tangible item record on the player's profile.
Evidence: relic_hunt.py handle_input() creates Item objects on relic collection; escort_mission.py run_pygame_session() creates and appends the Escort Medal on cooperative win.

Subsystem 5 — Permissions / Sharing Model
Source: Sonja Muretta (individual assignments)

Where in repo: guildquest.py — Permission enum, Campaign.shares dict, campaign_access()

What changed: Reused directly with no structural changes. The two permission levels (VIEW and COLLABORATIVE) and the shares dictionary on Campaign were taken exactly from Sonja's design. The campaign_access() method encapsulates the three-level role check (owner / editor / viewer) and is called everywhere the GUI needs to enforce access control.

Where it's used: The Share Campaign panel in the Campaigns page, access checks before editing or deleting campaigns, and the event creation flow which checks can_edit before allowing a new event to be added. This is also relevant to the co-op mini-adventure model conceptually — the COLLABORATIVE permission level mirrors the co-op design of EscortMission where both players have equal access to the shared campaign state.

Evidence: campaign_access() in guildquest.py; _share_camp(), _create_camp(), _add_event() in guildquest_gui.py all call this method.

Subsystem 6 — Settings / User Preferences
Source: Both Thang Nguyen and Sonja Muretta

Where in repo: guildquest.py — Settings dataclass, Theme and TimeDisplay enums

What changed: Sonja contributed the theme and time display preferences. Thang contributed the current realm selection. These were merged into a single Settings dataclass on the User object. The format_time_for_user() method was added to GuildQuestGame to apply the user's time display preference when rendering quest event times.

Where it's used: The Settings page in the GUI lets each user configure their theme, time display mode, and current realm. The sidebar shows the active user's current realm and world time at all times. format_time_for_user() is called when displaying event start and end times in the Quest Events page, so each user sees times formatted according to their personal preference (world clock only, realm local, or both).

Evidence: Settings dataclass and format_time_for_user() in guildquest.py; Settings page in guildquest_gui.py; sidebar _update_sb() reads from active user's settings.

6. Mini adventures

6.1 Relic Hunt (Competitive)

File: relic_hunt.py
Type: Competitive — first to 5 relics wins.
Realm: Avalon (looked up from GameContext at init time).

Gameplay

- 10 x 10 grid arena. 10 relics scattered randomly at the start.
- Players alternate turns. Each turn: one player moves one cell (WASD / Arrow Keys).
- Landing on a relic automatically collects it. First to 5 relics wins.
- Players cannot occupy the same cell simultaneously.

Subsystem Reuse

- Realm: GameContext.get_realm_by_name('Avalon') sets the scene.
- Item: each collected relic becomes a real Item(name='Relic #N', rarity=RARE) appended to the player's inventory_snapshot via profile.update_snapshot().
- PlayerProfile: win/loss recorded via on_player_win() / on_player_loss() hooks.

UI

Rendered in a Tkinter Toplevel window with a dark-themed Canvas. Player 1 is a blue circle labelled '1'; Player 2 is a red circle labelled '2'; relics are gold diamonds. Score panel updates live. Winner overlay shown on game end.

6.2 Escort Across the Realm (Co-op)

File: escort_mission.py
Type: Co-operative — both players win or lose together.
Realm: Shadowfen (looked up from GameContext at init time).
Requires: pip install pygame

Gameplay

- Real-time 2D platformer. Player 1 (orange P) is the Carrier; Player 2 (blue W) is the Partner.

- Carrier must reach the goal (G) in the top-right area of the map.
- Red hazard tiles cost 1 health on contact. Carrier starts with 3 health.
- Controls: P1 = A D W-jump; P2 = Arrow keys + Up-jump. Both move simultaneously.

Subsystem Reuse

- Realm: Shadowfen realm from ctx.realms is used to name the level.
- WorldClock: each hazard hit advances the world clock by 10 minutes (time cost), tying real-time gameplay to Sonja/Thang's time subsystem.
- Item: completing the escort grants both players a Legendary 'Escort Medal' added to their inventory snapshots.
- PlayerProfile: co-op win/loss recorded via on_complete() / on_player_loss().

UI

Runs in a separate Pygame window (960x540). Platforms, walls, hazards, and the goal are drawn in each frame. The main Tkinter window hides while Pygame runs and reappears when the player closes the window

7. How to add a new Mini Adventure

Adding a new mini-adventure to the GMAE requires changes to exactly two files and the creation of one new file. No changes to the game engine, GUI loop, or result tracking are needed.

Step 1 — Create your adventure file
Create a new Python file, e.g., timed_raid.py, in the same directory.

```
from mini_adventure import MiniAdventure, GameResult
from guildquest import PlayerProfile
from game_context import GameContext

class TimedRaid(MiniAdventure):
NAME = "Timed Raid"
DESCRIPTION = "Co-op: complete objectives before time runs out."
IS_COOPERATIVE = True

def init(self, ctx, p1, p2): ...
def get_instructions(self) -> str: ...
def get_state_view(self) -> str: ...
def handle_input(self, player_index, raw_input) -> str: ...
def advance_turn(self): ...
```

```
def is_over(self) -> bool: ...
def get_outcome(self) -> GameResult: ...
def reset(self): ...
```

Step 2 — Register in gmae.py Open gmae.py. At the top, add your import. In _register_adventures(), add one line:

```
from timed_raid import TimedRaid # add this import at top
def _register_adventures(self):
 self.registry.register(RelicHunt())
self.registry.register(TimedRaid()) # add this line
```

Step 3 — Add to the GUI adventure list
Open guildquest_gui.py. In _build_miniadv(), add a tuple to the adventures list:

```
adventures = [
("S Relic Hunt", "Competitive | ..."),
("S Escort Across the Realm", "Co-op | ..."),
("S Timed Raid", # ADD THIS
"Co-op | Complete objectives before the clock runs out."),
]
```

Step 4 — Add a launch case in _launch()
In _launch() in guildquest_gui.py, add an elif for your new index:

```
elif idx == 2: # index matches position in adventures list above
# For turn-based Tkinter adventures:
w = YourGameWindow(self.root, self.game,
u1.profile, u2.profile,
on_close=self._adv_closed)
w.grab_set()
# For real-time Pygame adventures: call engine.run_pygame_session()
```

Step 5 — Test

Run the app. Your adventure will automatically appear in the Mini-Adventures tab. The result tracker, profile updates, and the recent results panel all work without any additional changes.

Summary: 1 new file + 3 small edits (import, register, UI card + launch case). The interface enforces a clear contract. The registry and result tracker handle everything else automatically.

8. Design Retrospective

8.1 How the Design Evolved

The original individual assignments from Thang and Sonja were standalone CLI programs — each a single Python file with a menu loop, domain models, and simple in-memory data. There was no notion of a 'game', players, or mini-adventures. The first step in the final project was merging the two codebases into a single guildquest.py. This was not a trivial task: both implementations had overlapping classes (Campaign, QuestEvent, Realm, User) with slightly different field names and method signatures. We resolved this by taking Thang's structural naming as the base and overlaying Sonja's sharing/permissions model and WorldClock logic.

The GMAE layer (MiniAdventure interface, GameContext, GameController, registry) did not exist in either original design. It was designed from scratch for this project, with the Strategy pattern chosen deliberately to satisfy the extensibility requirement. The original design document proposed Java-style .java files; this was changed to pure Python once the team confirmed everyone was comfortable with Python. The GUI was originally planned as a simple Tkinter notebook. During development it was redesigned to use a sidebar navigation pattern, which gave more room for content and felt more like a real application. The dark fantasy colour theme was added to match the GuildQuest world aesthetic.

8.2 High Points

- The MiniAdventure interface worked exactly as intended — RelicHunt and EscortMission were both plugged in without any changes to the controller.
- The GameContext injection approach made subsystem reuse clean and explicit. Each adventure declares exactly which GuildQuest systems it touches through the context object.
- The dark-themed Tkinter GUI turned out significantly better than expected for a standard-library UI toolkit.
- Having 10 seeded realms, 4 characters, and 10 quest events made the GuildQuest management pages feel like a real product rather than a demo.
- The Relic Hunt game is genuinely playable and competitive — the randomised relic placement keeps each game different.

8.3 Low Points & Challenges

- Python 3.14 compatibility with Pygame was a blocker. The Escort Mission mini-adventure requires Python 3.11 or earlier due to Pygame's build dependencies. This was discovered late and required workaround documentation
- Merging two codebases that were designed independently (but for the same domain) took longer than anticipated. Several field names had to be normalised (e.g. 'userId' vs 'user_id', 'realmId' vs 'realm_id').
- The switch from Java (as referenced in the early design doc) to Python mid-project meant some design pattern descriptions had to be rewritten for Python idioms (no interfaces, ABC instead).
- Tkinter has limited styling capabilities compared to web frameworks. Achieving a polished dark theme required significant manual widget configuration rather than a simple stylesheet.
- Coordinating two mini-adventures with very different runtime models (turn-based Tkinter vs real-time Pygame) required the EscortMission wrapper to hide the difference behind the MiniAdventure interface.

8.4 What We Would Do Differently

- Define the MiniAdventure interface and GameContext first, before writing any adventure code. In this project, relic_hunt.py was partially written before the interface wa finalised, requiring some refactoring.
- Use a database (even SQLite) instead of JSON for persistence. The flat JSON approach works for profiles but would not scale to campaigns and events.
- Pin Python version requirements earlier. Discovering pygame's Python 3.14 incompatibility at the end of the project was disruptive.
- Write unit tests for the MiniAdventure lifecycle (init → loop → outcome → reset) as part of the interface definition, so all adventures are verified correct by construction.

9. GEN AI Deliverables

Prompt 1: "Look over all the files in our GuildQuest project (guildquest.py, guildquest_gui.py, mini_adventure.py, game_context.py, relic_hunt.py, escort_mission.py) and find anything that can be changed to make the code more functional, complete, or better aligned with the project requirements. Do not rewrite everything, just identify specific issues and suggest targeted improvements."

Full AI Response:

After reviewing all six files in the GuildQuest project, I found the following issues and suggested improvements:

In guildquest.py, the seed_data() method has a duplicate campaign ID. Both arc2 and arc3 are assigned the integer 102, which means when they are inserted into the campaigns dictionary, arc3 silently overwrites arc2. Only three campaigns will exist at runtime instead of four. Fix: assign arc3 the ID 103 and set next_campaign_id to 104.

Also in guildquest.py, the next_campaign_id counter is set to 102 at the top of **init** but seed_data() creates campaigns with IDs 100, 101, 102, and 103. This means the counter starts at a value that would immediately collide with an existing campaign if a user created a new one right away. Fix: set next_campaign_id to 104 after seeding.

In relic_hunt.py, the advance_turn() method switches the current player before checking win conditions. This means that after Player 1 collects their fifth relic, advance_turn() flips the current player to Player 2 and then checks scores, which assigns the correct outcome but leaves the turn indicator pointing at the wrong player for the final state display. The game still ends correctly but the final get_state_view() call will show the wrong player's turn in the status line. Fix: check win conditions before switching the current player, or check after but restore the player index so the final display is accurate.

In escort_mission.py, the run_pygame_session() method accesses self._p1.character_name and self._p2.character_name early in the method body. If run_pygame_session() is called before init() has been called, self._p1 and self._p2 will be None and this will throw an AttributeError with a confusing message. Fix: add a guard at the top of run_pygame_session() that checks whether init() has been called and raises a clear RuntimeError if not.

In guildquest_gui.py, the Mini-Adventures tab player dropdowns (_ref_miniadv) only populate with users who already have a saved profile. On first launch, before any profiles are created, both dropdowns are empty and there is no explanation of why. A new user would not know they need to go to the Player Profiles page first. Fix: add a short instructional label below the dropdowns that says something like "Players must have a saved profile. Go to Player Profiles to create one."

In guildquest_gui.py, the _on_adventure_closed callback refreshes the profiles page and the mini-adventures tab but does not refresh the dashboard. If a player completes a mini-adventure and then navigates to the dashboard, the stat counters and event list will be stale until they navigate away and back. Fix: call self._ref_dashboard() inside _adv_closed() as well.

In game_context.py, the item_pool attribute is defined as a Dict[str, Item] but nothing in the current codebase populates it. It is effectively dead code at the moment. Either populate it with the seeded items from GuildQuestGame at GameContext construction time, or remove it and document that it is a placeholder for a future loot system.

What we changed from this response:

We fixed the duplicate campaign ID in seed_data() by assigning arc3 the ID 103 and setting next_campaign_id to 104. This was a real defect that would have caused silent data loss at runtime. We added a guard at the top of run_pygame_session() that raises a RuntimeError with a clear message if called before init(). We added an instructional label to the Mini-Adventures tab directing users to the Player Profiles page if their dropdowns are empty. We added a _ref_dashboard() call inside _adv_closed() so the dashboard stays current after a game ends.

We did not change the advance_turn() win-check timing in relic_hunt.py. The AI correctly identified the technical issue but in practice it is not perceptible to players — the game ends immediately after advance_turn() returns and the final state view is drawn once with the correct outcome banner overlaid on the canvas, which hides the status line. Restructuring the input/advance loop to fix this would add complexity without any visible benefit to the player experience.

We did not remove the item_pool field from GameContext. We kept it as a documented placeholder because it is a natural extension point for a future loot or crafting system, and removing it would require updating the constructor calls in both mini-adventures.

Why we made these decisions:

The duplicate ID fix was non-negotiable — it was a silent correctness bug. The run_pygame_session() guard cost one line and prevented a confusing crash that would be hard to debug. The UI label addressed a real confusion point we had observed during our own testing sessions. The dashboard refresh was a small quality-of-life fix that made the application feel more consistent. The decision to skip the advance_turn() restructure reflects a judgment call

that the engineering cost outweighed the benefit — the AI flag was technically correct but practically irrelevant given how the final state is rendered.

Prompt 2:

"Here is our final updated design document for the GuildQuest Mini-Adventure Environment. Review it and identify places where the content could be improved, is inconsistent with the actual code, or is missing information that the assignment rubric requires. Focus on the UML section, the design patterns section, the reuse checklist, and the retrospective."

Full AI Response:

After reviewing the design document against the rubric requirements and the actual codebase, I found the following gaps and inconsistencies:

The UML diagram on page one shows only the GuildQuest backend domain layer. It correctly captures WorldTime, WorldClock, Realm, Campaign, QuestEvent, Character, Item, User, Settings, PlayerProfile, and GuildQuestGame. However the entire GMAE layer is absent. The MiniAdventure abstract class, GameContext, GameController, MiniAdventureRegistry, ResultTracker, and GameResult enum are not represented anywhere in the diagram. More critically, the inheritance relationship between MiniAdventure and its two concrete implementations — RelicHunt and EscortMission — is not shown. The Strategy pattern is the central architectural decision of the entire final project and the primary evidence of extensibility. Without a UML representation of this hierarchy, a reader cannot understand the core design from the diagram alone. The assignment says "UML as appropriate" and this is exactly where a UML diagram is most appropriate.

The design patterns section lists five patterns but does not include specific file and class references for most of them. The rubric explicitly states patterns must be identified "with file/class references." For example, the Strategy pattern entry says it is implemented in the mini-adventure files but does not name the specific class or method. The Input Validation security pattern does not cite the specific regex pattern or the method it appears in. Every pattern entry should include at minimum the filename and class name, and ideally the method name, where the pattern is implemented.

The retrospective section describes the final design well but does not fulfill the assignment requirement to explain how and why the design evolved from the original design document. The original design document proposed Java files (the pattern section referenced gmae/MiniAdventure.java, gmae/adventures/RelicHunt.java), a notebook-style Tkinter GUI

with tabs, and did not include a dashboard page or a sidebar navigation layout. None of these differences are mentioned in the retrospective. The assignment specifically asks for a comparison between what was originally proposed and what was actually built, including the reasons for each change. The current retrospective answers what the final design is, not how it got there.

The valid reuse checklist table in section six lists all subsystems correctly but the "Evidence" column for most entries says things like "see guildquest.py" without citing specific methods. The rubric asks for screenshots, demo notes, unit tests, or a short walkthrough as evidence. The entries should be more specific — for example, citing relic_hunt.py handle_input() as the specific location where Item objects are created on relic collection, or citing escort_mission.py run_pygame_session() as where the Escort Medal is granted.

The Gen-AI section is currently a placeholder. The rubric warns explicitly that undisclosed AI use may result in academic misconduct findings. This section must be completed before submission with all prompts, full responses, what was changed, and the reasoning behind those changes.

The document references the Escort Mission requiring pygame but does not acknowledge the Python 3.14 compatibility issue that was encountered during development. This is a significant real-world challenge that belongs in the retrospective under low points and challenges, and it also affects the instructions for running the project.

What we changed from this response:

We added a second UML section to the document covering the GMAE layer. This included a table showing the MiniAdventure abstract class with all eight interface methods, GameContext with its attributes and properties, GameController, MiniAdventureRegistry, ResultTracker, and GameResult. We also added an explicit note showing the inheritance hierarchy: MiniAdventure as the abstract base, with RelicHunt and EscortMission as concrete implementations.

We updated every design pattern entry to include the specific filename and method name where the pattern appears. For example, the Input Validation entry now cites relic_hunt.py handle_input() re.compile(r'^[wasdWASD]$') specifically. The Strategy pattern entry now cites mini_adventure.py as the interface, relic_hunt.py and escort_mission.py as the concrete strategies, and guildquest_gui.py _launch() as the context that selects and runs them.

We expanded the retrospective to explicitly compare the original design document against the final implementation. This included noting that the original design proposed Java, the final implementation is Python; the original proposed a tabbed notebook GUI, the final uses a sidebar layout; the original did not include a dashboard page, the final does; and the original did not anticipate the Pygame compatibility problem with Python 3.14.

We added more specific method citations to the reuse checklist evidence column, pointing to exact methods in exact files rather than just filename references.

We completed the Gen-AI section with these two entries.

Why we made these decisions:

The GMAE UML addition was necessary because the Strategy pattern is the most important design decision in the project and could not be understood from the existing diagram. A reader evaluating the system's extensibility needs to see the interface and the inheritance hierarchy to assess whether the design actually achieves that extensibility. The pattern-specific file citations were required by the rubric and we had simply omitted them in the first draft adding them cost no design effort and directly addressed a rubric criterion. The retrospective expansion required us to go back to the original design document and compare it honestly against what we built, which surfaced some meaningful differences (Java to Python, tab to sidebar, no dashboard to dashboard) that are worth discussing as evidence of design evolution. We treated the AI's feedback on the Gen-AI placeholder as a reminder rather than a suggestion it was always going to be completed, and these two entries are the result.